

IPRO/PRM3

Principes **algorithmiques** et programmation

Introduction

*Qu'est ce qu'un **algorithme** ?*

Informatique ?

Informatique = Science de l'information
En fait, traitement automatisé de l'information.

Computer science = Science de l'ordinateur

Compute (to)	=	calculer
Computation	=	calcul informatisé
Computational	=	automatisable

Qu'est ce qu'un ordinateur ?

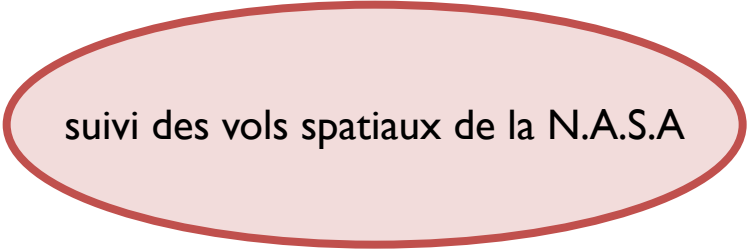
De manière quelque peu simpliste, nous pourrions répondre :

un ordinateur est une machine

pouvant effectuer « des **tâches** intellectuelles de routine »
en exécutant très rapidement des **opérations simples**.



l'addition ou la comparaison de deux nombres.



suivi des vols spatiaux de la N.A.S.A

Algorithme ?

Un ordinateur ne peut naturellement accomplir que les tâches spécifiées dans les termes des opérations simples qu'il est capable de réaliser.

On doit lui indiquer ...

comment la tâche doit être menée à bien,
quelles opérations il doit accomplir et
dans quel ordre.

On doit lui donner une **marche à suivre**.

Cette description est appelée un **algorithme**.

Donnée(s) → **Algorithme** → Résultat(s)

Définition classique :

Un **algorithme** est une suite de règles à appliquer dans un ordre déterminé à un nombre fini de **données** pour arriver, en un nombre fini d'étapes, à un certain **résultat**. Chaque **opération** doit être exécutable et avoir un sens non ambigu.

Un algorithme est la description d'une méthode systématique et progressive pour résoudre un problème.

Un ordinateur est donc une machine à qui nous allons FAIRE FAIRE des opérations qui l'amèneront finalement à résoudre des problèmes.

```

alimentation de la réserve
mise à jour du numéro de la semaine en cours
mise en service et affichage de message "EN SERVICE"
tant que la réserve n'est pas vide
| attendre l'introduction de la carte
| lire les informations enregistrées sur la carte
| si la carte est périmée
| | alors restituer la carte avec message "CARTE PERIMEE"
| | sinon
| | | recommencer
| | | demander à l'utilisateur son code
| | | comptabiliser le nombre d'essais
| | tant que le code est incorrect et que le nombre d'essais < 3
| | si le code est correct
| | | alors
| | | | calculer le nombre de billets autorisés
| | | | demander à l'utilisateur le nombre de billets désirés
| | | | tant que le nbre de billets désirés est > au nbre de billets autorisés
| | | | | afficher le message "PLAFOND ATTEINT"
| | | | | demander le nombre de billets désirés
| | | | |
| | | | mettre à jour les informations sur la carte
| | | | mettre à jour la réserve de billets
| | | | sortir les billets et la carte
| | | sinon
| | | | conserver la carte avec le message "CARTE CONSERVEE"
|
mise hors service et affichage du message "HORS SERVICE"

```

Instructions simples

```

alimentation de la réserve
mise à jour du numéro de la semaine en cours
mise en service et affichage de message "EN SERVICE"
tant que la réserve n'est pas vide
    attendre l'introduction de la carte
    lire les informations enregistrées sur la carte
    si la carte est périmée
        alors restituer la carte avec message "CARTE PERIMEE"
    sinon
        recommencer
        demander à l'utilisateur son code
        comptabiliser le nombre d'essais
        tant que le code est incorrect et que le nombre d'essais < 3
            si le code est correct
                alors
                    calculer le nombre de billets autorisés
                    demander à l'utilisateur le nombre de billets désirés
                    tant que le nbre de billets désirés est > au nbre de billets autorisés
                        afficher le message "PLAFOND ATTEINT"
                        demander le nombre de billets désirés
                    mettre à jour les informations sur la carte
                    mettre à jour la réserve de billets
                    sortir les billets et la carte
            sinon
                conserver la carte avec le message "CARTE CONSERVEE"
    mise hors service et affichage du message "HORS SERVICE"

```

Les instructions simples manipulent des données, spécifient les opérations élémentaires à exécuter.

Instructions structurées

```

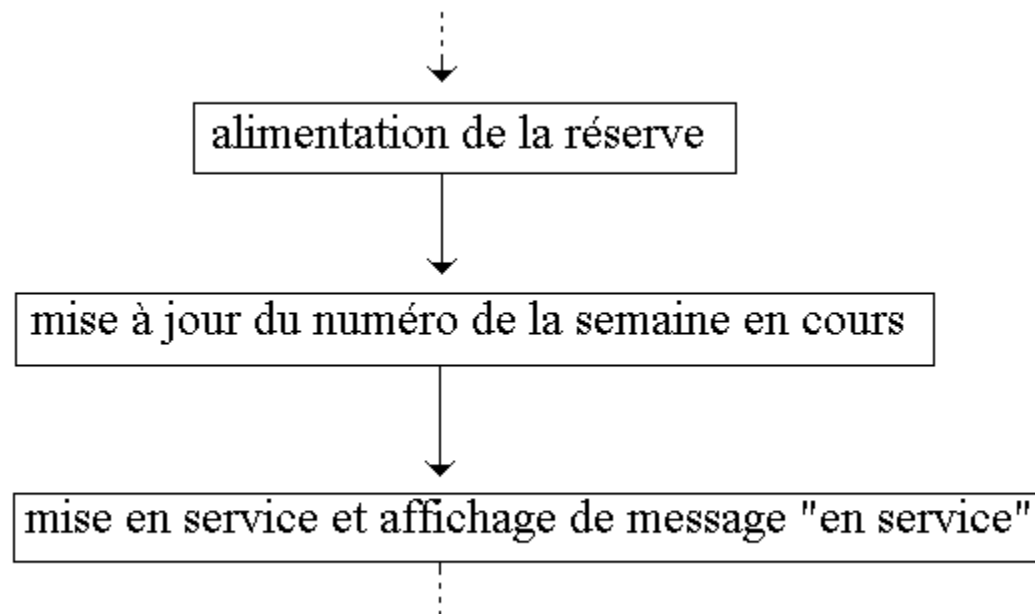
alimentation de la réserve
mise à jour du numéro de la semaine en cours
mise en service et affichage de message "EN SERVICE"
tant que la réserve n'est pas vide
    attendre l'introduction de la carte
    lire les informations enregistrées sur la carte
    si la carte est périmée
        alors restituer la carte avec message "CARTE PERIMEE"
        sinon
            recommencer
            demander à l'utilisateur son code
            comptabiliser le nombre d'essais
            tant que le code est incorrect et que le nombre d'essais < 3
                si le code est correct
                    alors
                        calculer le nombre de billets autorisés
                        demander à l'utilisateur le nombre de billets désirés
                        tant que le nbre de billets désirés est > au nbre de billets autorisés
                            afficher le message "PLAFOND ATTEINT"
                            demander le nombre de billets désirés
                        mettre à jour les informations sur la carte
                        mettre à jour la réserve de billets
                        sortir les billets et la carte
                    sinon
                        conserver la carte avec le message "CARTE CONSERVEE"
            mise hors service et affichage du message "HORS SERVICE"

```


Structure de contrôle : séquence

Les structures de contrôle sont là pour « contrôler » le déroulement de l'algorithme.

Séquence

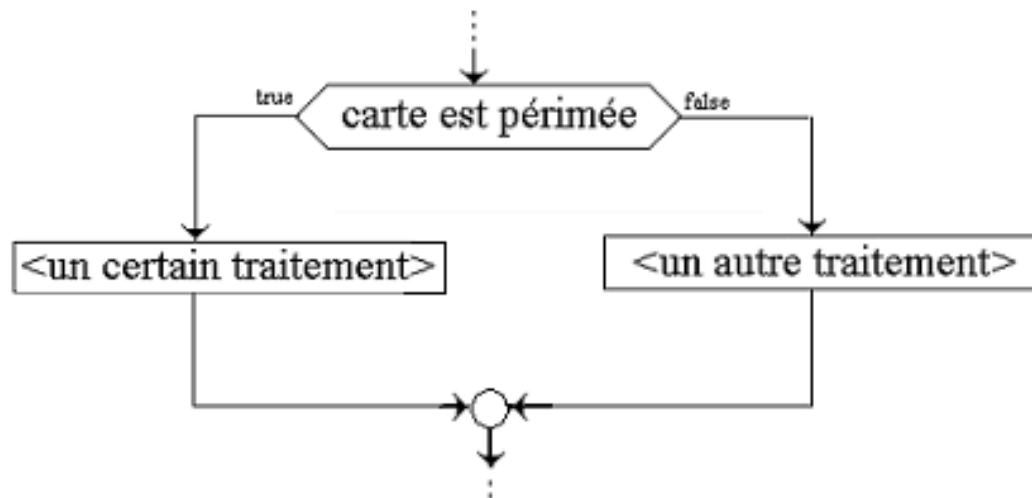


Structure de contrôle : alternative

Les structures de contrôle sont là pour « contrôler » le déroulement de l'algorithme.

Alternative

Si la carte est périmée
alors <un certain traitement>
sinon <un autre traitement>
fsi

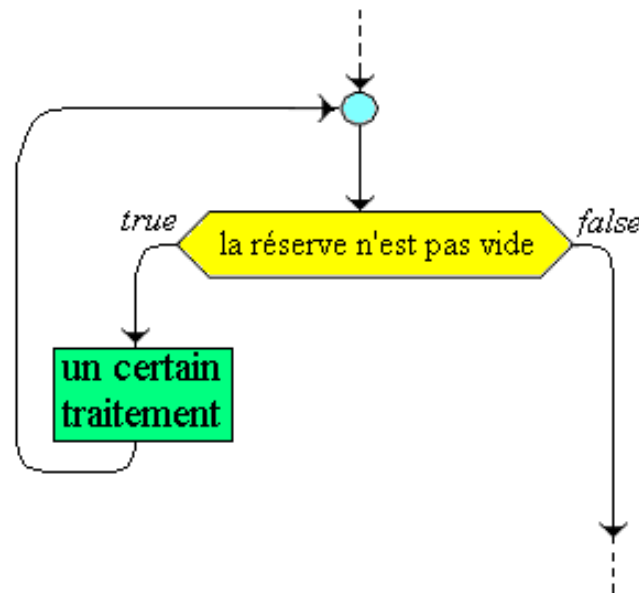


Structure de contrôle : répétitive

Les structures de contrôle sont là pour « contrôler » le déroulement de l'algorithme.

Répétitive

tant que la réserve n'est pas vide *faire*
<un certain traitement>
ftant



Structure de contrôle : répétitive

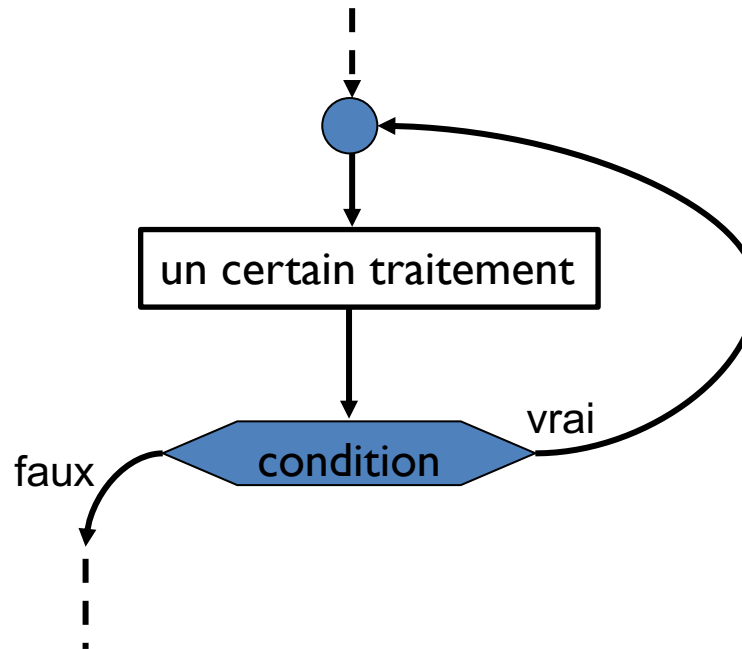
Les structures de contrôle sont là pour « contrôler » le déroulement de l'algorithme.

Répétitive

recommencer

<un certain traitement>

tant que le code est incorrect et que le nombre d'essais < 3



Conclusions

Un **algorithme** est une combinaison
par imbrication ou concaténation
de structures de traitement.

IPRO/PRM3

Principes algorithmiques et **programmation**

Introduction

*Qu'est ce qu'un **programme** ?*

Programme et langage de programmation

Pour qu'un algorithme soit correctement exécuté par le processeur (l'entité responsable d'exécuter le programme), il convient que ce dernier soit capable de l'interpréter.

Lorsque le processeur est un ordinateur, l'algorithme doit être exprimé sous la forme d'un **programme**, qui est écrit dans un **langage de programmation**

Un programme consiste donc en une **suite d'instructions**, chacune d'elles spécifiant les opérations que l'ordinateur doit exécuter. La nature des instructions d'un programme dépend du langage de programmation.

Langage machine

Comme les circuits électroniques de chaque ordinateur ne peuvent reconnaître et exécuter directement qu'un nombre très limité d'instructions, tout programme doit être, avant son exécution, converti pour n'être exprimé qu'avec ces instructions dites élémentaires.

Celles-ci sont du genre :

- additionner deux nombres

- voir si un nombre est égal à zéro

- changer d'emplacement mémoire une donnée

- ...

L'ensemble des instructions exécutables directement par un ordinateur forme un langage appelé le **langage machine**.

Langages

Élaboré à partir de l'alphabet binaire (0,1), le langage machine est particulier à chaque machine.

Se posent alors des problèmes tels que lisibilité et portabilité.

De plus, ces langages machine sont si primitifs - *chaque instruction y est tellement élémentaire qu'elle n'exprime qu'une infime partie d'un algorithme* - qu'il est extrêmement pénible et fastidieux de les utiliser.

Pour rendre la programmation plus facile, des **langages de plus en plus évolués** ont été développés.

Une instruction dans ces langages peut correspondre à un ensemble d'instructions machine et exprime donc une étape plus importante d'un algorithme.

Traducteur : compilateur et interpréteur

Des programmes spécifiques, appelés **traducteurs de langage** ont pour tâche de traduire un programme écrit en langage de haut niveau en un programme compréhensible par la machine.

On parle de compilateur ou d'interpréteur (voir la nuance dans d'autres cours) : un **compilateur** Java par exemple.

La traduction consiste à transformer chaque instruction du programme évolué en une séquence équivalente d'instructions en langage machine.

*« Un compilateur sert le plus souvent à traduire un **code source** écrit dans un langage de programmation en un autre langage, habituellement un langage d'assemblage ou un **langage machine**. ».*

Conclusions

